

Progetto di Laboratorio di Reti

(Corso A)

Anno Accademico 2025/26

Corsi Leonardo - Matricola 656276

Introduzione

Il progetto si basa sul gioco “**Connection**” presente sul sito del NYT.

L'obiettivo è quello di indovinare set di 4 parole accomunati da una categoria comune, che può variare da forme grammaticali a temi veri e propri.

Gli utenti possono connettersi al server e dopo la registrazione/login accedere direttamente alla partita in corso, permettendo dunque la partecipazione al gioco insieme agli altri giocatori attivi.

Architettura

Il progetto implementa l'architettura **Client-Server** per l'implementazione del gioco, utilizzando il protocollo **TCP** per l'instaurazione della connessione tra Client e Server. Per la gestione delle notifiche asincrone viene utilizzato il protocollo **UDP**.

Scelte progettuali

NIO+ThreadPool Lato server

Il server usa **Java NIO** con un “Selector” per gestire tutte le connessioni TCP su un unico thread dedicato. Il Selector monitora tutti i canali registrati e segnala quando uno è pronto per la lettura.

Il messaggio viene quindi delegato a una **ThreadPool**, in questo caso FixedThreadPool, che lo processa in parallelo, così il Selector resta libero per continuare a gestire gli altri canali.

Configurazione tramite **.properties**

Sia il server che il client leggono i parametri da file **.properties** invece di accettarli come argomenti a riga di comando. Questo rende la modifica dei parametri strutturali più comoda togliendo di fatto la necessità di ricordare ogni volta la sintassi e i valori esatti del comando.

Persistenza JSON con Gson

I dati dei giocatori, la storia delle partite e il database delle parole sono salvati su file JSON, gestiti tramite la libreria **Gson**.

Le operazioni di salvataggio sono sincronizzate per evitare scritture concorrenti corrotte.

Notifiche via UDP Multicast

Le notifiche ai client (nuova partita iniziata, classifica finale, server in shutdown) vengono inviate via **UDP multicast**. Il multicast permette di raggiungere tutti i client con una sola trasmissione, evitando l'iterazione su tutte le connessioni TCP.

Sessione per connessione

Ogni connessione TCP ha un oggetto **Session** dedicato che mantiene lo stato del giocatore durante la partita (username, categorie indovinate, numero di errori, flag di fine partita). Questo consente anche il recupero dello stato dopo un logout/login nella stessa partita.

Schema dei thread

[Lato server]

- ❖ **Main Thread**: Inizializza le strutture dati, carica i dati dai file .json e fa partire il loop del selector NIO, accettando connessioni e leggendo i messaggi in arrivo, delegando poi alla ThreadPool
- ❖ **FixedThreadPool(20)**: Pool da 20 thread che processa le richieste dei client
- ❖ **GameTimer Thread**: Gestisce il ciclo delle partite: avvio, pausa e transizione alla partita successiva;
Inoltre interagisce con i *Session* utente per resettare lo stato dei giocatori alla fine di ogni partita e inviare la classifica via multicast.
- ❖ **ShutDown Hook**: Rimane in attesa della combinazione di tasti "CTRL+C", dopodiché avvia la funzione di **shutdown** del server fermando il pool e il timer, chiudendo i canali e notificando i client dello spegnimento del server.

[Lato Client]

- ❖ **Main Thread**: Avvia un loop interattivo, occupandosi di leggere l'input dell'utente, inviare le richieste TCP e mostrare le risposte
- ❖ **Multicast Listener Thread**: Thread demone che ascolta le notifiche UDP Multicast dal server e le stampa a schermo.

Strutture dati

- ❖ **PlayerManager:**
 - `ConcurrentHashMap<String, Player>`: Utilizzata per la mappa dei giocatori, in modo da consentire accesso concorrente ai thread.
 - `Set<String>`: Utilizzato per i giocatori online con supporto da parte della *ConcurrentHashMap* per il lookup.
- ❖ **GameManager:**
 - `ConcurrentHashMap<Integer, Game>`: Utilizzata per il caricamento delle partite disponibili all'avvio, permettendo letture concorrenti.
 - `ConcurrentHashMap<Integer, GameStats>`: Utilizzata per aggiornamenti concorrenti delle statistiche della partita.
- ❖ **ServerMain:**
 - `Set<Session>`: Utilizzata per l'iterazione (sicura) durante il reset delle partite.
- ❖ **GameHistoryManager:**
 - `ConcurrentHashMap<Integer, List<PlayerResult>>`: Utilizzata per aggiornamenti concorrenti dello storico delle partite.
- ❖ **Session:**
 - `HashSet<String>`: Utilizzata per le categorie indovinate durante la sessione, con accesso protetto da `synchronized`.
 - `ByteBuffer`: Buffer NIO dedicato per ogni connessione TCP
- ❖ **Player:**
 - `HashSet<String>`: Utilizzata per il recupero delle categorie attive e dunque dello stato dopo il login.
 - `int[]` (da 6 elementi): Utilizzata per l'istogramma errori, ovvero per le statistiche sul numero di errori per partita (0–5).

File di salvataggio

Sono stati definiti i seguenti file JSON per garantire la persistenza dei dati in caso di spegnimento del server:

- ❖ **Connections_Data.json**: File riservato alla memorizzazione dei giochi che vengono caricati a runtime.
- ❖ **gameHistory.json**: File riservato alla memorizzazione (storico) delle partite.
- ❖ **players.json**: File riservato alla memorizzazione delle credenziali e delle statistiche personali degli utenti.

Primitive di sincronizzazione

Per garantire la thread-safety in un ambiente multi-threaded, sono state utilizzate:

ConcurrentHashMap

Usata per tutte le collezioni principali; garantisce letture non bloccanti e riduce la contesa tra thread.

Metodi synchronized

I metodi di *PlayerManager*, *GameManager* e *GameHistoryManager* che modificano lo stato condiviso e scrivono su file sono dichiarati **synchronized**. Questo ci garantisce che le operazioni di lettura/modifica/salvataggio non possono sovrapporsi tra thread diversi.

Synchronized su session

La gestione della proposta (*handleSubmitProposal*) è sincronizzata sull'oggetto *Session* del giocatore specifico. Questo evita **race condition** tra il thread che processa la proposta e il timer thread che resetta le sessioni a fine partita.

Volatile

Le variabili condivise tra thread che richiedono solo **visibilità** (non atomicità di operazioni composte) sono dichiarate *volatile*:

- ❖ **running**: Flag di shutdown, letto dal loop del Selector
- ❖ **currentGame**: Riferimento alla partita corrente, aggiornato dal timer
- ❖ **pause**: Flag di pausa tra una partita e l'altra
- ❖ **startTime**: Timestamp di inizio partita, usato per calcolare il timeout
- ❖ **lastCompletedGame**: ID dell'ultima partita completata, per il recovery al login

Compilazione ed esecuzione

Prerequisiti

Java 8+ (testato con OpenJDK 1.8.0_482)

- Librerie nella cartella <<lib/>> (già incluse nel repository):
 - gson-2.13.1.jar → serializzazione/deserializzazione JSON.

Compilazione

Dalla directory radice del progetto (**bash**):

—javac -cp "lib/gson-2.13.1.jar:." src/*.java -d bin/—

Su Windows sostituire ":" con ";" nel classpath.

Esecuzione

Per la creazione dei **.jar** (**bash**)

- ❖ jar cvfm Server.jar server-manifest.txt -C bin/ . (server)
- ❖ jar cvfm Client.jar client-manifest.txt -C bin/ . (client)

Per eseguire i **.jar** (**bash**)

- ❖ java -jar Server.jar
- ❖ java -jar Client.jar

NOTA: avviare prima il server del client e in terminali separati!

Non sono necessari parametri a riga di comando: tutti i dati di configurazione sono già presenti nei file **.properties** forniti

Sintassi operazioni

Una volta avviato il **Client**, seguire le istruzioni a schermo. Il Menù presentato al client si differenzia se l'utente è logged in o meno.

Se non ancora autenticato:

- ❖ **register**: permette la registrazione di un nuovo utente
- ❖ **login**: permette l'autenticazione di un utente
- ❖ **updateCredentials**: permette l'aggiornamento delle credenziali di un utente
- ❖ **exit**: chiude il terminale (in alternativa premere CTRL+C)

Se autenticato:

- ❖ **play**: permette la scelta delle 4 parole da inviare
- ❖ **stats**: mostra le statistiche del giocatore
- ❖ **leaderboard**: mostra
 - la classifica dei top (k) utenti (scelta 1)
 - il rank di un giocatore specifico (scelta 2)
- ❖ **gameinfo**: mostra
 - tempo rimanente, le proposte corrette, le parole rimaste ancora da raggruppare correttamente, il numero di errori ed il punteggio corrente. (scelta -1)
 - La corretta assegnazione delle 16 parole ai 4 gruppi, il numero di proposte corrette, il numero di errori fatti, ed il punteggio ottenuto in caso di partita conclusa per vittoria, sconfitta o tempo scaduto (scelta = id partita richiesta)
- ❖ **gamestats**: mostra
 - tempo rimanente per quella partita, il numero di giocatori con quella partita ancora in corso, inumero di giocatori che hanno concluso la partita ed il numero di giocatori che hanno concluso la partita con una vittoria. (scelta -1);
 - il numero di giocatori che hanno partecipato alla partita, il numero di giocatori che hanno concluso la partita, il numero di giocatori che hanno concluso la partita con una vittoria ed il punteggio medio dei giocatori che hanno partecipato alla partita in caso di partita conclusa. (scelta = id partita richiesta);
- ❖ **logout**: permette la disconnessione del proprio profilo dal server
- ❖ **exit**: chiude il terminale (in alternativa premere CTRL+C)